



第10章 对象和类

张仁斌@合肥工业大学软件学院

本章主要内容



10.1 面向过程程序设计与面向对象程序设计



10.2 类和对象

10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）

10.6 对象数组

10.7 对象指针和this指针

10.8 作业与实践

面向过程的程序设计



◆面向过程的程序设计（**Procedure Oriented Programming, POP**）是一种编程范式，其中程序被分解为一系列**步骤或过程**（也称为**函数或子程序**），**每个过程执行特定的任务**

◆面向过程的程序设计的一般步骤

■分析问题

- 理解并分析问题，明确目标，将问题分解成几个独立的子任务或步骤

■定义过程

- 针对每个子任务，定义一个过程（函数或子程序），每个过程应有明确的输入和输出，这样它们可以独立地被测试和调用

■实现过程

- 使用编程语言实现每个过程，确保每个过程都按预期工作，并且代码清晰、可读

■组织调用

- 在程序的主函数中按顺序调用这些过程，以完成整个任务，确保适当地传递参数和接收返回值

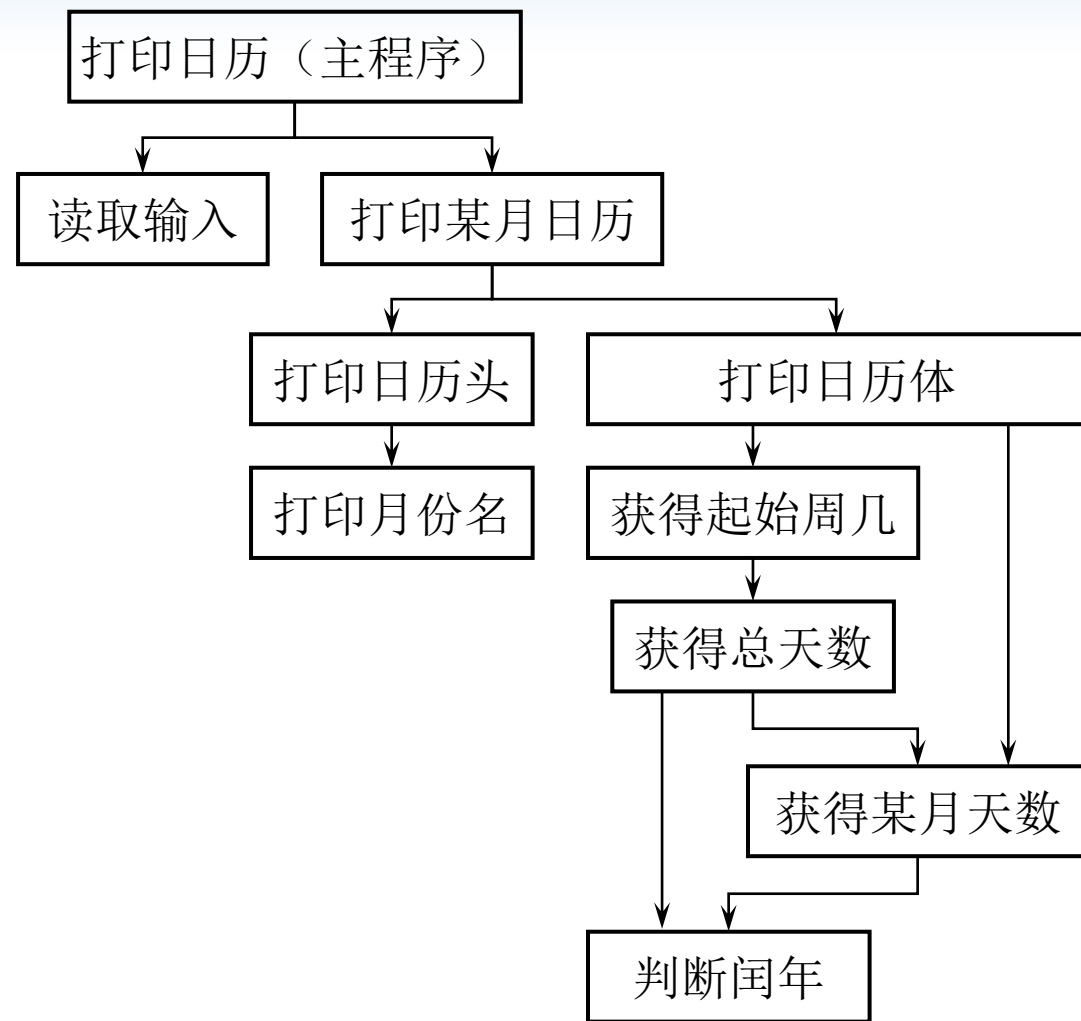


◆面向过程的**结构化程序设计**是采用面向过程的方法来设计结构化程序

■结构化程序通常包含一个**主过程**和**若干个子过程**，其中每个子过程都描述某个子问题的解决方法，再由主过程**自顶向下**调用各子过程，逐步解决整个问题，如图所示，整个执行过程是从主过程开始，再在主过程的结束语句处结束

- 以功能为核心，基于过程，其本质是**功能分解细化**
- 数据和函数分开，功能封装，重用性差、维护困难，安全性差，且不适合解决复杂问题

■用户**需求的变化**大部分是针对功能的，因此，这种变化对于基于过程的设计来说是**灾难性**的



不适用于复杂系统的开发

OO (Object Oriented) 技术的历史、现状



◆ 面向对象是一种对现实世界理解和抽象的方法

- 起初，“面向对象”是**专指**在**程序设计中**采用**封装**、**继承**、**多态**等设计方法
- 现在，面向对象的思想已经涉及到**软件开发的各个方面**，例如，面向对象的分析（OOA, Object Oriented Analysis），面向对象的设计（OOD, Object Oriented Design）、**面向对象的编程实现**（**OOP**, Object Oriented Programming）

◆ 历史

- 60年代末：Simula 67（第一个面向对象的语言，获图灵奖）
- 80年代初：Smalltalk（第二个面向对象的语言，第一个IDE）
- 80年代中：C++、OOP、OOD和OOA进入软件开发实践
- 90年代：OOP、OOD和OOA获得广泛应用

◆ 现状

- OOP、OOD和OOA成为最重要的软件开发方法
- OO在分布计算、数据库、系统软件等领域大显身手

面向对象的程序设计

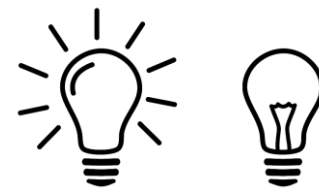


◆对象（Object）

- 对象是对客观事物的**抽象**，是人们要进行研究的任何事物，它不仅能表示具体的事物，还能表示抽象的规则、计划或事件，例如某个电灯

◆对象的状态和行为

- 对象具有**状态**，对象用数据值来描述它的状态（例如灯的亮、灭）
- 对象具有**行为**，用于改变对象的状态，由函数定义（例如开灯、关灯）
- 对象实现了**数据和操作的结合**，使**数据和操作封装于对象**的统一体中



◆类（Class）

- 类是具有相同类型的对象的**抽象**，例如某种类型电灯，因此，对象的抽象是类，类的具体化是对象，也可以说类的**实例**（instance）是对象
- 类具有**属性**，属性是对象的**状态**的抽象，用**数据结构**来描述类的属性
- 类具有**操作**，操作是对象的**行为**的抽象，用**操作名和实现该操作的方法**来描述



◆ 类与对象的关系

- 类是对某一类事物的描述，是**抽象**的，是一类事物的**共性**部分
- 对象是一类事物的**实例**，是**具体**的，是实际存在的该类事物的**个性**个体
- 类是对象的模板，对象是类的实例

◆ 程序员用类建立对象，软件系统是由多个对象相互作用构成的

◆ 面向对象程序设计**的重点是类的设计**，不是对象的设计

面向对象的特征



◆面向对象具有**三个基本特征：封装、继承和多态**

◆封装

■封装是面向对象编程的重要**原则**，指隐藏类的内部数据和实现细节（使用者不必了解细节），只暴露必要的接口供外部访问。封装通过访问public、private、protected等修饰符实现

■封装的特点

- 信息隐藏：封装将类的内部实现细节隐藏起来，外部只能通过接口访问类的属性和方法，不关心实现细节
- 安全性增强：通过限制外部访问，可以防止不合理的修改和操作，提高代码的安全性
- 易维护：封装使代码的实现细节与外部接口分离，使代码更加模块化，便于维护和升级



◆ 继承

■ 当继承是面向对象编程中的一种重要关系，它允许一个类（**子类**、**派生类**）继承另一个类（**父类**、**基类**）的属性和方法。通过继承，子类可以**复用**父类的代码，并可以在此基础上添加、修改或覆盖特定的行为

■ 继承的特点

- 代码复用：继承允许子类重用父类的代码，避免了重复编写相同的代码
- 派生与扩展：子类可在父类的基础上进行扩展，添加新的属性和方法，从而实现新的功能
- 层次结构：继承关系可以形成类的层次结构，使得代码更加有组织、易于理解和维护
- 多态支持：继承是多态的基础，通过子类对象可以实现对父类方法的重写，实现不同对象的不同行为



◆多态

■多态是面向对象编程的一个重要概念，它允许不同类的对象以适合自身的方式响应相同的消息，表现出不同的行为、产生不同的结果，这种现象称为多态。多态通过方法的**重写（重载）**和基类引用指向**派生**类对象来实现

■多态的特点

- 统一接口：多态允许使用相同的接口来调用不同类的对象，提供了一种统一的调用方式
- 代码重用：多态通过方法的重写，使得不同类可以共享相同的接口和方法名，减少了重复编写代码
- 动态绑定：多态的方法调用在运行时动态决定，而不是在编译时固定，使得程序更加灵活

POP与OOP的对比



◆ 面向过程的程序设计

- 程序=算法 + 数据结构

- 数据结构和算法相分离，所以，系统庞大后，其控制、移植、重用就成了问题

◆ 面向对象的程序设计

- 对象=数据结构+算法

- 程序=对象 + 对象 + + 消息

- 消息（message）的作用就是对对象的控制。程序设计的关键是设计好每个类以生成对象，及确定向这些对象发出的命令，使各对象完成相应操作

使用OOP的原因



- ◆ 顺应人类思维习惯，让软件开发人员在解空间中直接模拟问题空间中的对象及其行为（POP是计算思维，从计算机角度分析）
- ◆ 改善软件结构（模块化与封装），提高软件灵活性
- ◆ 支持软件重用
- ◆ 支持增量式开发，支持大型软件开发

本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象



10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）

10.6 对象数组

10.7 对象指针和this指针

10.8 作业与实践



类的定义

◆形式

class 类名标识符

{

private:

访问权限

只能被类成员函数访问

私有数据和函数（私有成员）

public:

类对象可以访问的数据和函数

公有数据和函数（公有成员，外部接口）

protected:

保护型数据和函数（保护成员）

};

◆示例：时钟类的定义

```
class Clock
```

```
{
```

```
    public:           // 公有成员函数，外部接口
```

```
        void setTime(int h=0, int m=0, int s=0);
```

```
        void showTime();
```

```
    private:         // 私有数据成员
```

```
        int hour, minute, second;
```

```
};
```

```
class A{  
    int x;  
    void m();  
};
```

```
class B{  
    int x;  
    public:  
        void m();  
};
```

说明：

①类是一种用户自定义的数据类型

②类中定义的数据（变量）和函数（行为操作），称为**数据成员和函数成员**

③**class**为关键字，后跟类名

④**访问控制**属性：公有（**public**）、私有（**private**）和保护（**protected**），缺省定义为**private**，体现封装性、数据隐藏（紧跟类名后面声明私有成员，可省略**private**）

类的成员函数



◆ 成员函数的声明与实现

■ 类内声明

■ 类外实现，形式为：

返回值类型 **类名::**函数成员名(形参列表)

```
{  
    函数体  
}
```

域限定符，哪个
类的成员函数

```
class Clock
```

```
{
```

```
    public:
```

```
        void setTime(int h=0, int m=0, int s=0);
```

```
        void showTime();
```

```
    private:
```

```
        int hour, minute, second;
```

```
};
```

类内声明，可带默认的形参值

```
void Clock::setTime(int h, int m, int s)
```

```
{
```

```
    hour = h;
```

```
    minute = m;
```

```
    second = s;
```

```
}
```

```
inline void Clock::showTime()
```

```
{
```

```
    cout << hour << ":" << minute << ":" << second << endl;
```

```
}
```



◆ 内联成员函数

■ 隐式声明：将函数体直接放入类的主体内

➤ 例如：

```
class Test
{
    private:
        int x, y;
    public:
        void input( ) { cin >> x >> y;}
        void output();
};
```

■ 显式声明：实现函数时用关键字inline声明

➤ 例如：

```
inline void Test::output( )
{
    cout << "面积为: " << x*y << endl;
}
```

类的成员函数用于实现某种操作，成员函数的定义体可以在类的声明体中，也可以在类的声明体外

在类声明体中实现的函数是内联函数。在类声明体外实现的函数可以通过在函数定义上加上inline来表示该函数是内联的，否则不是内联函数

在类的声明体内定义成员函数的**优点**是使整个类集中于程序代码的同一位置上；**缺点**是增加了类声明的规模和复杂性，而且内联的函数代码并不被相同类的对象所共享，因而增大了程序的内存开销

引例：类定义和类实现的分离



◆ 时钟类完整示例（类的定义与实现，存储在**同一文件**中）

// 时钟类的定义

```
class Clock
{
public:    // 公有成员函数，外部接口
    void setTime(int h=0, int m=0, int s=0);
    void showTime();
private: // 私有数据成员
    int hour, minute, second;
};
```

// 时钟类成员函数的具体实现

```
void Clock::setTime(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
}
```

// 显示时间

```
inline void Clock::showTime()
{
    cout << hour << ":" << minute << ":" << second << endl;
}

int main()
{
    Clock clock;           // 定义对象 clock
    cout << "First time set and output:" << endl;
    clock.setTime( );        // 设置时间为默认值
    clock.showTime( );
    cout << "second time set and output:" << endl;
    clock.setTime(8, 30, 30); // 设置时间为 8:30:30
    clock.showTime( );
    cout << "sizeof(clock): " << sizeof(clock) << endl;
    return 0;
}
```

类的数据成员所占内存大小

D:\Edu-CPP\chap10\ex10-01.exe

```
First time set and output:
0:0:0
second time set and output:
8:30:30
sizeof(clock): 12
```

类定义和类实现的分离



◆ C++ 允许将类的定义和实现分离

须新建工程 (project)
以管理、编译多文件

■ 类定义描述类的“约定”，而类实现则实现这一约定

- 类定义简单列出所有数据成员、函数成员原型，类实现给出函数成员的实现，两者可以分别存储在两个分离的**同名但扩展名不同**的文件中
- 类定义文件的扩展名为**.h**，类实现文件的扩展名为**.cpp**

Clock.h 文件内容

```
#ifndef CLOCK_H
#define CLOCK_H

class Clock {
public:
    void setTime(int h=0, int m=0, int s=0);
    void showTime();
private:
    int hour, minute, second;
};
#endif
```

避免被多次include
时导致重复定义

Clock.cpp 文件内容

```
#include "Clock.h"
#include <iostream>
using namespace std;

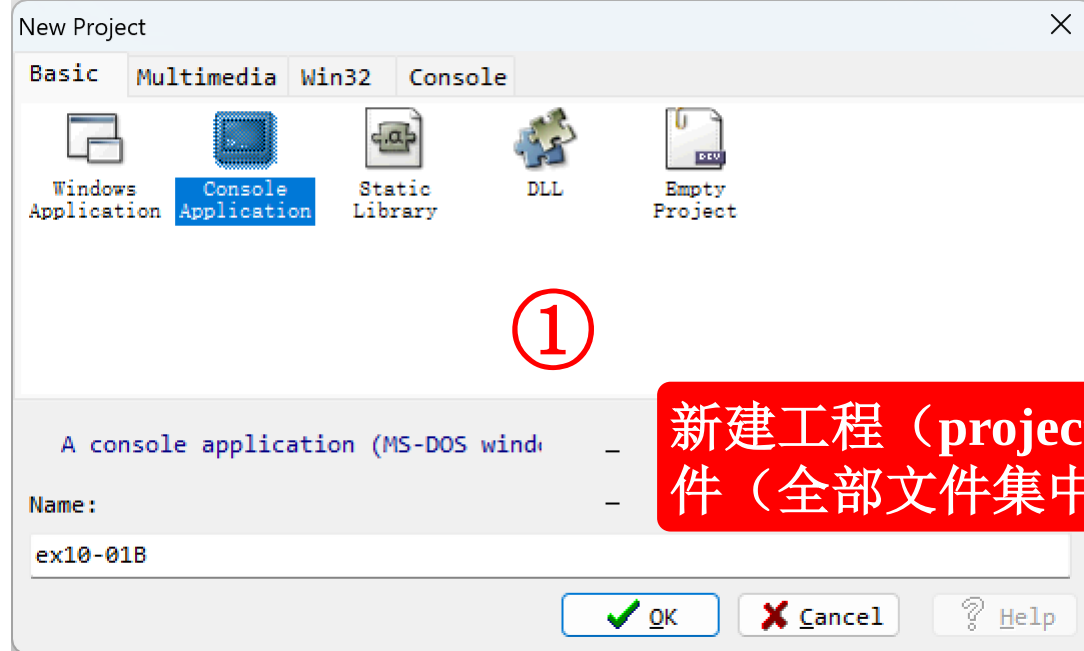
void Clock::setTime(int h, int m, int s) {
    hour = h;
    minute = m;
    second = s;
}

void Clock::showTime() {
    cout << hour << ":" << minute << ":" << second << endl;
}
```

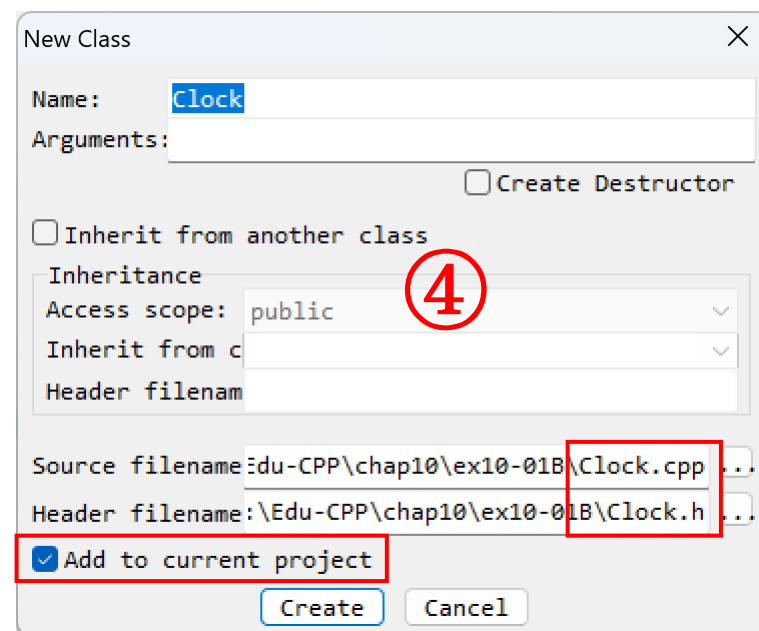
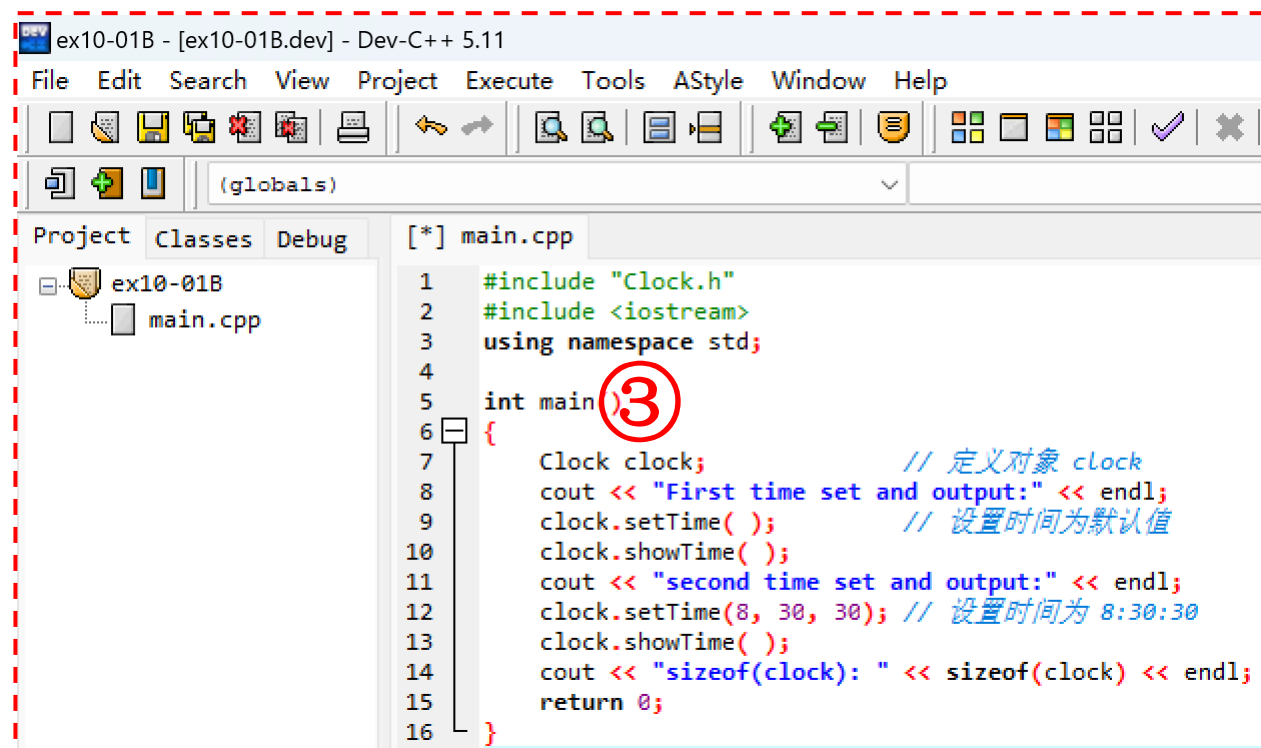
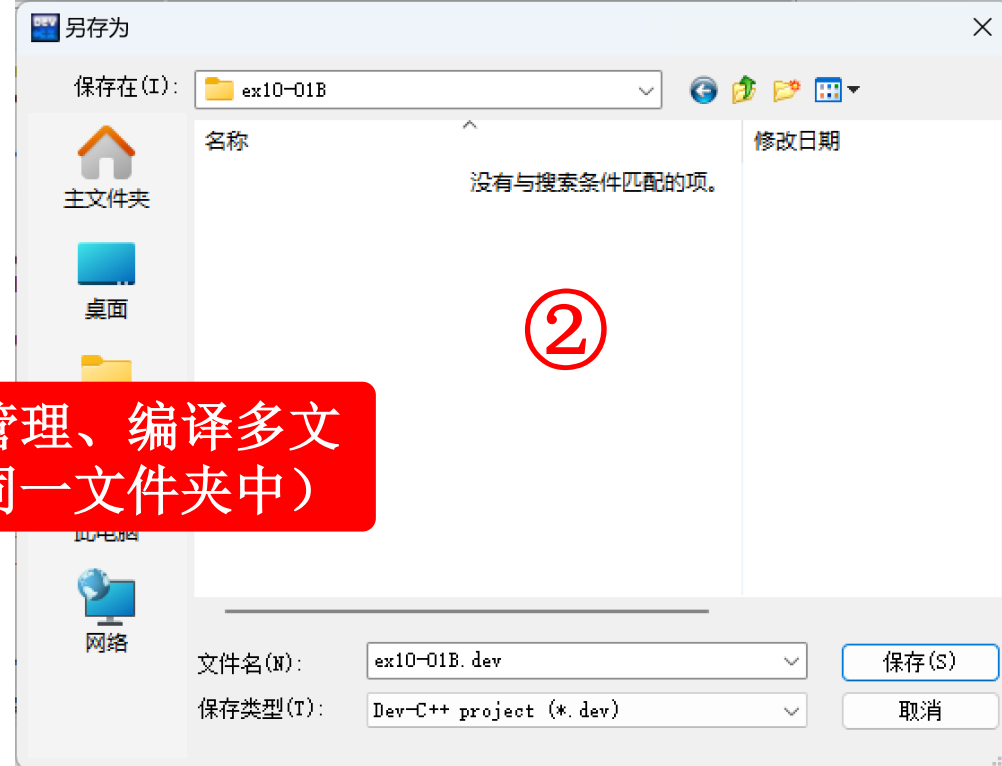
main.cpp 文件内容

```
#include "Clock.h"
#include <iostream>
using namespace std;

int main()
{
    Clock clock;
    .....
    return 0;
}
```



新建工程（project）以管理、编译多文件（全部文件集中放在同一文件夹中）



ex10-01B - [ex10-01B.dev] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

Project Classes Debug main.cpp [*] Clock.h [*] Clock.cpp

ex10-01B

- main.cpp
- Clock.h
- Clock.cpp

```
1 #ifndef CLOCK_H
2 #define CLOCK_H
3
4 class Clock {
5     public:
6         void setTime(int h=0, int m=0, int s=0);
7         void showTime();
8     private:
9         int hour, minute, second;
10 };
11
12 #endif
```

5

ex10-01B - [ex10-01B.dev] - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

(globals)

Project Classes Debug main.cpp [*] Clock.h [*] Clock.cpp

ex10-01B

- main.cpp
- Clock.h
- Clock.cpp

```
1 #include "Clock.h"
2 #include <iostream>
3 using namespace std;
4
5 void Clock::setTime(int h, int m, int s)
6 {
7     hour = h;
8     minute = m;
9     second = s;
10 }
11
12 void Clock::showTime()
13 {
14     cout << hour << ":" << minute << ":" << second << endl;
15 }
16
```

6



对象的创建与使用

◆ 类是一种用户自定义的数据类型，与定义普通变量类似，定义对象的形式为：**类名 对象名**；

■ 例如：

```
Clock clock, clock2;  
Clock *p = &clock;  
Clock array[5];
```

只分配用于存储数据成员的内存，成员函数**代码**放于公共区域中，为每个对象共享

◆ 对象成员的访问

对象名.公有成员函数名(参数表)

对象名.公有数据成员

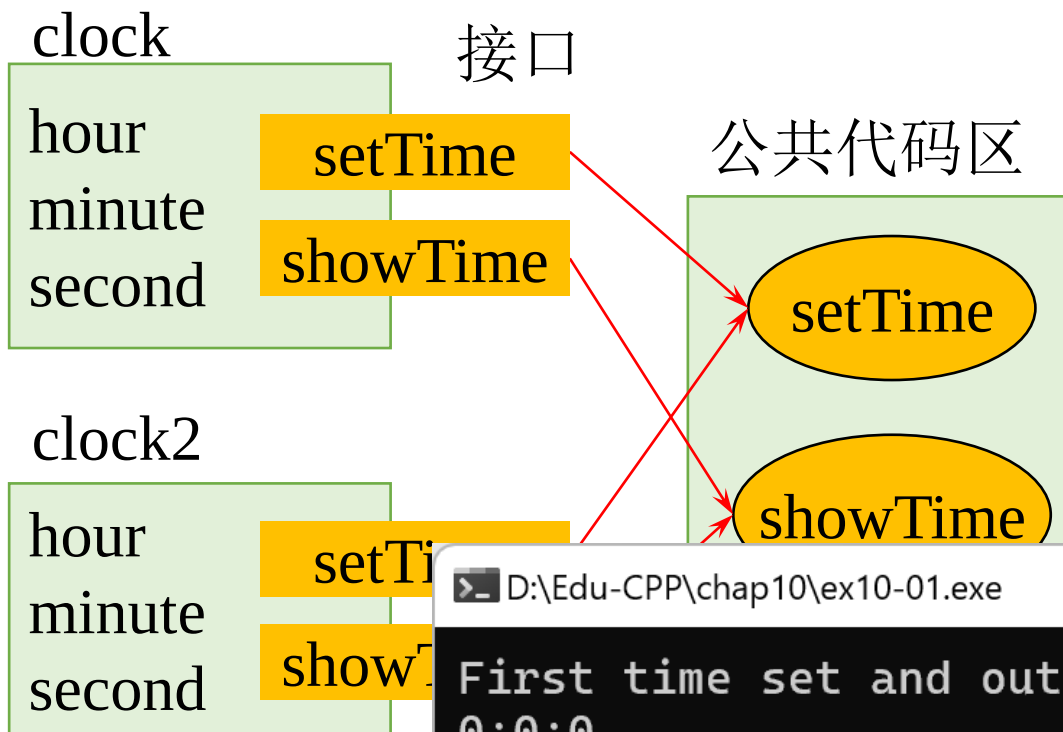
■ 例如：

成员访问运算符

```
clock.setTime(8, 30, 30);
```

```
(*p).setTime( );
```

```
p->setTime( );
```



D:\Edu-CPP\chap10\ex10-01.exe

```
First time set and output:  
0:0:0  
second time set and output:  
8:30:30  
sizeof(clock): 12
```

类成员的访问控制



◆ 公有成员

- 声明了类的外部接口

clock.setTime();

clock.hour = 12; ❌

类外不能访问私有成员，若变更为public，才可如此操作

- 类对象 **只允许** 访问 **公有成员**，这体现了类的 **封装** 功能

◆ 私有成员

- 只允许本类成员函数访问，而类外部的任何访问都是非法的

◆ 保护成员

- 和私有类型的性质相似，其差别在于继承过程中对产生的新类影响不同

```
class Clock
{
    public:
        void setTime(int h=0, int m=0, int s=0);
        void showTime();
    private:
        int hour, minute, second;
};

Clock clock;
```

```
void Clock::setTime(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
}
```

类成员可以访问该类的其他成员

示例：求矩形的面积和周长



◆ 采用面向过程的程序设计思想实现——设计流程：输入→计算→输出

// 定义变量

double length, wide, area, perimeter;

// 输入

cout << "请输入长和宽：";

cin >> length >> wide;

// 计算

area = length * wide;

perimeter = 2 * (length + wide);

// 输出

cout << "面积为： " << area << endl;

cout << "周长为： " << perimeter << endl;

顺序执行

double rectArea(double length, double wide)

```
{  
    return length * wide;  
}
```

double rectPerimeter(double length, double wide)

```
{  
    return (length + wide) * 2;  
}
```

int main() {

double length, wide, area, perimeter;

cout << "请输入长和宽：";

cin >> length >> wide;

cout << "面积为： " << rectArea(length, wide) << endl;

cout << "周长为： " << rectPerimeter(length, wide) << endl;

return 0;

}

函数(模块化)

D:\Edu-CPP\chap10\ex10-02.exe

```
请输入长和宽： 2 3  
面积为： 6  
周长为： 10
```



◆采用面向对象的程序设计思想实现——设计矩形类

■描述矩形属性的数据：长、宽

■描述矩形行为的函数：接受输入，计算面积，计算周长

```
class Rectangle {  
private:  
    double length, wide;  
public:  
    void input() {  
        cout << "请输入长和宽: ";  
        cin >> length >> wide;  
    }  
    double getArea() {  
        return length * wide;  
    }  
    double getPerimeter() {  
        return 2 * (length + wide);  
    }  
};
```

```
int main() {  
    Rectangle rect;  
    rect.input();  
    cout << "面积为: " << rect.getArea() << endl;  
    cout << "周长为: " << rect.getPerimeter() << endl;  
    return 0;  
}
```

用户不需了解
类的实现细节

D:\Edu-CPP\chap10\ex10-02C.exe

```
请输入长和宽: 2 3  
面积为: 6  
周长为: 10
```


本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象

10.3 构造函数和析构函数



10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）

10.6 对象数组

10.7 对象指针和this指针

10.8 作业与实践

构造函数（Constructor）



◆变量的初始化

```
int i = 10;  
int k (10);
```

◆创建类对象时如何初始化？

```
Clock clock;
```

◆构造函数

■定义：和**类名同名**的**成员函数**称为构造函数

■作用：给对象进行初始化

■说明

- 不能指定返回类型（void也不行），一般是公有的
- 一个类中可定义**多个构造函数**（重载），带或不带参数，可默认参数值
- 一个类中如果没定义构造函数，则编译器自动建立一个无参构造函数（**默认构造函数**），一旦定义了构造函数则不提供默认构造函数
- 构造函数在创建对象时被**自动调用，不能显式调用**

```
class Clock  
{  
    public:  
        void setTime(int h=0, int m=0, int s=0);  
        void showTime();  
    private:  
        int hour=0, minute=0, second=0; ✗  
};
```



◆有参构造函数

```
class Clock
{
public:
    Clock(int h, int m, int s);
    void setTime(int h=0, int m=0, int s=0);
    void showTime();
private:
    int hour, minute, second;
};

Clock::Clock(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
    cout << "有参构造函数被调用\n";
}
```

初始化列表：在构造函数参数列表后面加冒号，冒号后面对对象成员进行初始化，成员之间用逗号分隔

```
Clock:: Clock(int h, int m, int s):
    hour(h), minute(m), second(s)
{
    cout << "有参构造函数被调用\n";
}
```

```
void Clock::setTime(int h, int m, int s)
{
    hour = h;
    minute = m;
    second = s;
}

inline void Clock::showTime()
{
    cout << hour << ":" << minute << ":" << second << endl;
}

int main()
{
    Clock clock(1, 2, 3);
    clock.showTime( );
    return 0;
}
```

D:\Edu-CPP\chap10\ex10-03.exe

有参构造函数被调用
1:2:3



默认构造函数是在类中没有定义任何构造函数时，由编译器自动生成的一个没有参数的特殊构造函数，它并不执行任何操作，仅用于确保对象在创建时处于已知状态

◆ 无参构造函数

■ 若类仅有带参数的构造函数，没有无参数构造函数，则无法创建**不带参数的对象**

➤ 只要类中定义了构造函数，编译器则不提供**默认构造函数**

```
class Clock
{
public:
    Clock(int h, int m, int s);
    Clock( ){ cout << "无参构造函数被调用\n"; } // 无参构造函数
    .....
private:
    int hour, minute, second;
};
```

方案1

```
int main( )
{
    Clock c;
    c.showTime();
    return 0;
}
```



```
class Clock
{
public:
    Clock(int h=0, int m=0, int s=0);
    .....
private:
    int hour, minute, second;
};
```

方案2

基于ex10-03，通过编译执行，演示没有无参构造函数时Clock c;的报错情况，及两种方案的编译执行



◆ 引例：创建一个新的对象时，如何利用已有的对象初始化该新对象？

■ 调用什么构造函数？

◆ 拷贝构造函数（复制构造函数）

■ 形如 **`X::X(X &)`** 或 **`X::X(const X &)`** 的构造函数

■ 用一个已存在的对象初始化一个新的同类对象

拷贝构造函数必须以
引用方式传递参数

```
int main( )
{
    Clock c1(8, 10, 0);
    Clock c2(c1);
    c1.showTime( );
    c2.showTime( );
    return 0;
}
```



D:\Edu-CPP\chap10\ex10-04.exe

```
有参构造函数被调用
拷贝构造函数被调用
8:10:0
8:10:0
```

```
class Clock
{
public:
    Clock(int h, int m, int s);
    Clock( );           // 无参构造函数
    Clock(Clock& c);    // 拷贝构造函数
    .....
private:
    int hour, minute, second;
};
```

`Clock::Clock(Clock& c)`

```
{
    hour = c.hour; minute = c.minute; second = c.second;
    cout << "拷贝构造函数被调用\n";
}
```



◆在C++中，下面三种对象需要调用拷贝构造函数

- 一个对象用于给另外一个对象进行初始化（常称为赋值初始化）
- 一个对象作为函数参数，以值传递的方式传入函数体
- 一个对象作为函数返回值，以值传递的方式从函数返回（编译器默认优化编译将导致不调用拷贝构造函数）

◆拷贝构造函数必须以引用的形式传递参数，避免传参过程调用构造函数

◆编译器自动生成的默认拷贝构造函数的功能是把初始值对象的每个数据成员的值逐一复制到新建立的对象中

构造函数示例



```
class Point {
public:
    Point(int xx=0, int yy=0); // 有参（默认值）构造函数
    Point(Point &p);           // 拷贝构造函数
    int getX() {return x;}
    int getY() {return y;}
private:
    int x, y;
};

Point::Point(int xx, int yy) {
    x = xx; y = yy;
    cout << "有参构造函数被调用(" << x << ", " << y << ")" << endl;
}

Point::Point(Point &p) {
    x = p.x; y = p.y;
    cout << "拷贝构造函数被调用(" << x << ", " << y << ")" << endl;
}
```



void f1(**Point** p) { 传引用，避免调用构造函数，更高效

```
    cout << "函数f1内部(" << p.getX() << ", " << p.getY() << ")" << endl;
}
```

①创建对象时一定会调用构造函数

②根据定义对象带的参数不同，自动调用相应的构造函数

③如果没有成功调用构造函数，就不能创建对象

```
Point f2(int x, int y) {
    cout << "函数f2内部(" << x << ", " << y << ")" << endl;
    Point t(x, y);
    return t; // 编译器优化编译将导致不调用拷贝构造函数
}
```

```
int main() {
    Point a(4, 5), b;
    Point c(a); // 调用拷贝构造函数
    Point d = b; // 调用拷贝构造函数
    cout << "d1(" << d.getX() << ", " << d.getY() << ")" << endl;
    d = a; // 赋值，不调用拷贝构造函数
    cout << "d2(" << d.getX() << ", " << d.getY() << ")" << endl;
    f1(b); // 调用拷贝构造函数
    b = f2(8, 9); // 调用拷贝构造函数？没有
    cout << "b(" << b.getX() << ", " << b.getY() << ")" << endl;
    return 0; }
```

初始化时调用拷贝构造函数

D:\Edu-CPP\chap10\ex10-05.exe

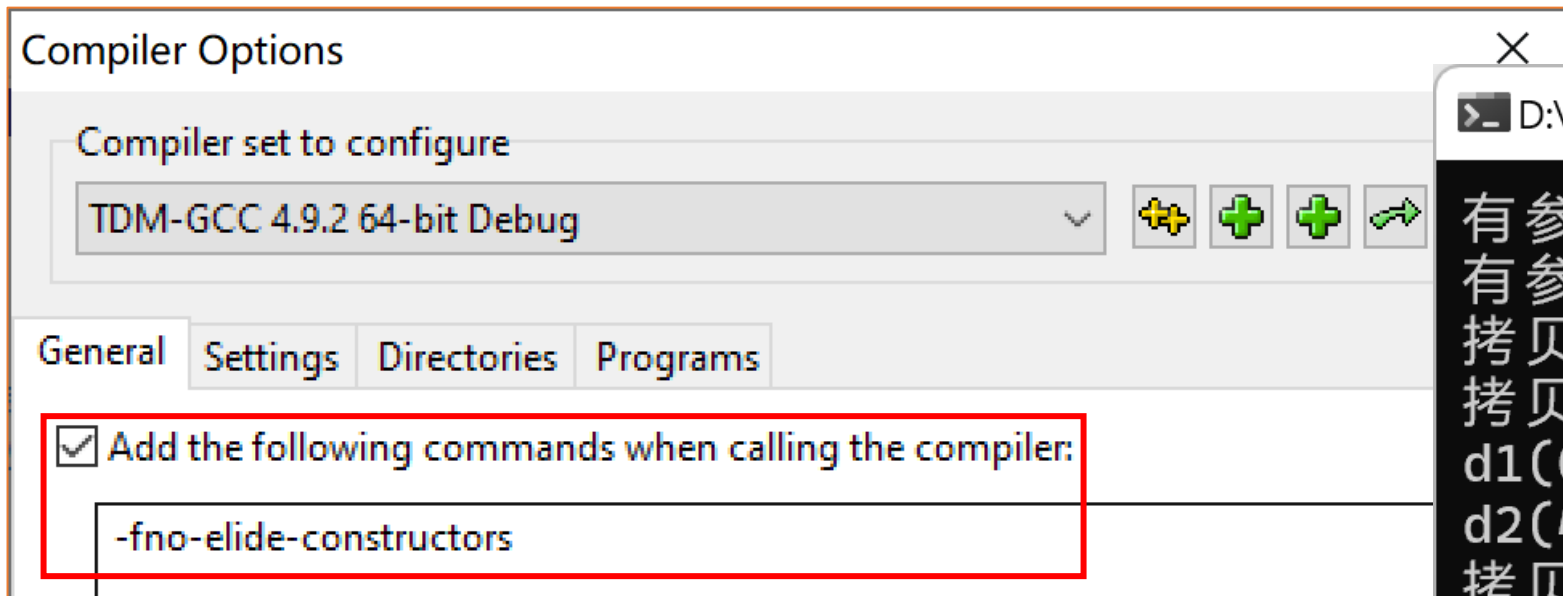
```
有参构造函数被调用(4, 5)
有参构造函数被调用(0, 0)
拷贝构造函数被调用(4, 5)
拷贝构造函数被调用(0, 0)
d1(0, 0)
d2(4, 5)
拷贝构造函数被调用(0, 0)
函数f1内部(0, 0)
函数f2内部(8, 9)
有参构造函数被调用(8, 9)
b(8, 9)
```




◆ 去掉编译器优化项，以便函数返回对象时调用拷贝构造函数

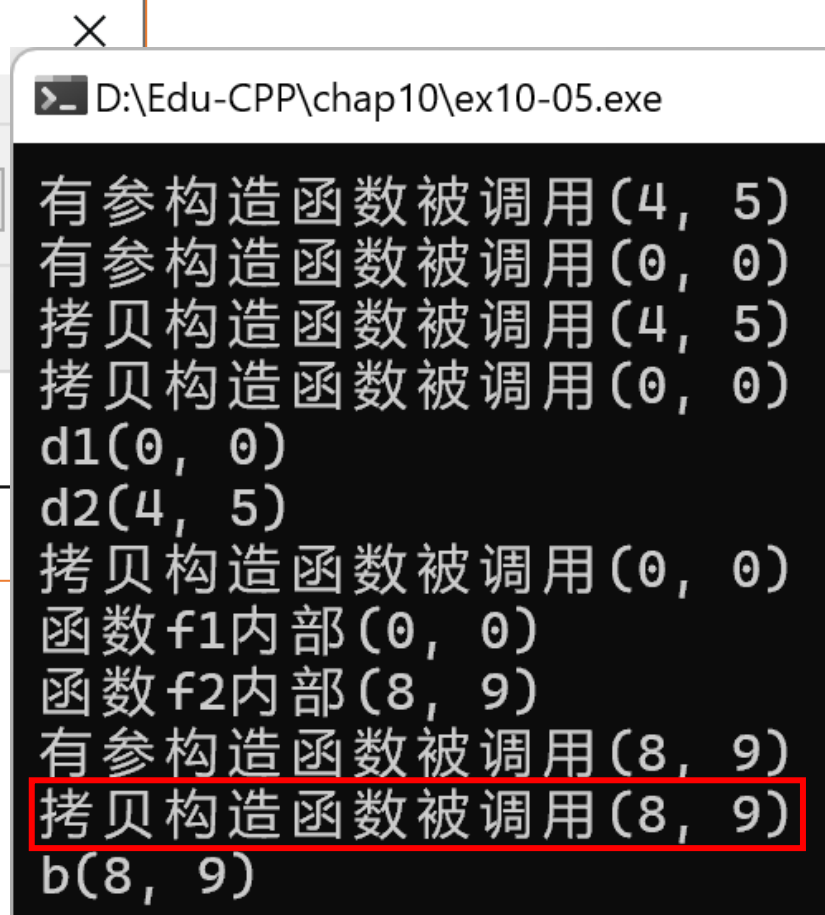
■ 添加编译选项 **-fno-elide-constructors**

考试时若遇到此问题，按添加了此选项关闭编译优化处理



调用拷贝构造函数生成临时对象接收f2()
的返回值（对象），然后再将其赋值给b

```
b = f2(8, 9); // 调用拷贝构造函数  
cout << "b(" << b.getX() << ", " << b.getY() << ")" << endl;
```





析构函数 (Destructor)

◆ 定义

- 与**类名同名**的**成员函数**，且之前冠以**波浪号~**，以区别于构造函数

◆ 作用

- 清理工作，例如释放由构造函数分配的内存等

◆ 说明

- 在对象撤消时（对象的生命期结束时），被**自动**调用，**不能**主动调用
- 没有任何返回值（void也不行）
- 不接受任何参数，是唯一的**无重载**的成员函数
- 若没有自定义析构函数，编译器将自动生成**缺省的析构函数**

```
class Clock {  
public:  
    Clock(int h, int m, int s);  
    ~Clock( );//析构函数  
    .....  
};
```

```
Clock::~~Clock( )  
{  
    cout << "析构函数被调用\n";  
}
```

析构函数调用时机示例1



```
class Test
{
private:
    int num;
public:
    Test( );
    ~Test( );
};

Test::Test( )
{
    num = 0;
}

Test::~~Test( )
{
    cout << "析构函数被调用" << endl;
}
```

```
int main()
{
    Test x;
    cout << "退出main函数" << endl;
    return 0;
}
```

> D:\Edu-CPP\chap10\ex10-06.exe

退出main函数
析构函数被调用

析构函数调用时机示例2



```
class MyClass {
private:
    int value;
public:
    MyClass(int i = 0) {value = i;}
    MyClass(MyClass &cp);
    void setValue(int i) {value = i;};
    void print() {cout << "This Print value=" << value << endl;};
    ~MyClass();
};

MyClass::MyClass(MyClass &cp) {
    value = cp.value;
    cout << "拷贝构造函数中value=" << value << endl;
}

MyClass::~MyClass( ) {
    cout << "析构函数中value=" << value << endl;
}
```

```
MyClass classTestFunc(MyClass obj) {
    cout << "In classTestFunc( )..." << endl;
    obj.print();
    obj.setValue(10);
    cout << "classTestFunc( )返回..." << endl;
    return obj;
}

void gFunc( ) {
    MyClass my(5), ret;
    cout << "即将调用classTestFunc( )..." << endl;
    ret = classTestFunc(my);
    cout << "In gFunc( )..." << endl;
    cout << "my==>"; my.print();
    cout << "ret==>"; ret.print();
    cout << "退出gFunc( )" << endl;
}

int main( ) {
    gFunc( );
    cout << "退出main( )" << endl;
    return 0; }
```



```
void gFunc( ) {  
    MyClass my(5), ret;  
    cout << "即将调用classTestFunc( )..." << endl;  
    ret = classTestFunc(my);  
    cout << "In gFunc( )..." << endl;  
    cout << "my==>"; my.print();  
    cout << "ret==>"; ret.print();  
    cout << "退出gFunc( )" << endl;  
}
```

```
MyClass classTestFunc(MyClass obj) {  
    cout << "In classTestFunc( )..." << endl;  
    obj.print();  
    obj.setValue(10);  
    cout << "classTestFunc( )返回..." << endl;  
    return obj;  
}
```

```
int main( ) {  
    gFunc( );  
    cout << "退出main( )" << endl;  
    return 0; }
```

函数形参传值，调用拷贝构造函数

调用拷贝构造函数生成临时对象接收函数返回对象

函数形参对象析构

接收函数返回值临时对象析构

形参传值不改变my

按堆栈先进后出，先析构后定义的ret

全部对象都在gFunc()中构建，并随之退出而析构

D:\Edu-CPP\chap10\ex10-07.exe

```
即将调用classTestFunc( )...  
拷贝构造函数中value=5  
In classTestFunc( )...  
This Print value=5  
classTestFunc( )返回...  
拷贝构造函数中value=10  
析构函数中value=10  
析构函数中value=10  
In gFunc( )...  
my==>This Print value=5  
ret==>This Print value=10  
退出gFunc( )  
析构函数中value=10  
析构函数中value=5  
退出main( )
```



匿名对象（临时对象）

◆ 临时使用，**只用一次**，或作为函数的实参，**用完即销毁**

```
class Some
{
int n;
public:
    Some(int i) {n = i; cout << "constructor!\n"; }
    Some(const Some &s) { n = s.n; cout << "copy!\n"; }
    ~Some( ) { cout << "destroy: n = " << n << "\n"; }
    int retN( ) { return n; }
};

int main(int argc, char* argv[ ])
{
    cout << Some(123).retN( ) << "\n";
    cout << "wait1...\n";
    Some s = Some(456);
    cout << "wait2...\n";
    return 0;
}
```

D:\Edu-CPP\chap10\ex10-08.exe

```
constructor!
123
destroy: n = 123
wait1...
constructor!
copy!
destroy: n = 456
wait2...
destroy: n = 456
```

本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象

10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝



10.5 类的组合（对象成员）

10.6 对象数组

10.7 对象指针和this指针

10.8 作业与实践

深拷贝与浅拷贝

◆浅拷贝

- 实现对象间数据元素的一一对应复制

◆深拷贝

- 当被复制的对象数据成员**有指针时**，不是复制该指针成员本身，而是将指针所指内容进行复制

D:\Edu-CPP\chap10\ex10-09.exe

```
zhang
zhang
wang  键盘输入temp_str为wang
wang
wang  为什么myname
      跟着yourname
      变wang了?
```

```
class Name
{
public:
    Name() { p = NULL; }
    Name(char *s);
    ~Name();
    void set(char *s) { strcpy(p, s); }
    void show() { cout << p << endl; }
private:
    char *p;
};
```

```
Name::Name(char *s)
{
    p = new char[strlen(s)+1];
    strcpy(p, s);
}

Name::~~Name( )
{
    cout << "Destructing." << endl;
    if( p != NULL) delete [] p;
}
```

```
int main() {
    char temp_str[20] = "zhang";
    Name myname(temp_str);
    Name yourname(myname);
    myname.show();
    yourname.show();
    cin >> temp_str;
    yourname.set(temp_str);
    myname.show();
    yourname.show();
    return 0;
}
```


原因分析



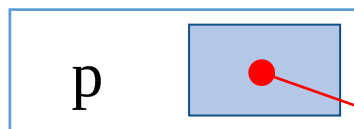
```
char temp_str[20] = "zhang";    temp_str wang
```

```
Name myname(temp_str);
```

```
Name yourname(myname);
```

```
Name::Name(char *s)
{
    p = new char[strlen(s)+1];
    strcpy(p, s);
}
```

myname



yourname



wang

// 编译器自动添加的拷贝构造函数

```
Name::Name(const Name& op)
{ p = op.p; }
```

浅拷贝

```
cin >> temp_str;
```

```
yourname.set(temp_str);
```

如果一个类包含指向动态存储空间指针类型的数据成员，则应为此类设计复制构造函数，实现深拷贝

深拷贝示例

深拷贝保证类对象之间的独立



```
#include<iostream>
#include<cstring>
using namespace std;

class Name
{
public:
    Name() { p = NULL; }
    Name(char *s);
    Name(const Name &other);
    ~Name();
    void set(char *s)
    {
        if(p != NULL) strcpy(p, s);
    }
    void show() { cout << p << endl; }
private:
    char *p;
};
```

```
Name::Name(char *s)
{
    p = new char[strlen(s)+1];
    strcpy(p, s);
}
```

Name::Name(const Name &other)

```
{
    if (other.p!=NULL)
    {
        p = new char[strlen(other.p) + 1];
        if(p != NULL) strcpy(p, other.p);
    }
    cout << "Copy constructing.\n";
}
```

使成员指针指向独立的新地址

```
Name::~~Name( ) {
    cout << "Destructing." << endl;
    if( p != NULL) delete [] p;
}
```

```
int main()
{
    char temp_str[20] = "zhang";
    Name myname(temp_str);
    Name yourname(myname);
    myname.show();
    yourname.show();
    cin >> temp_str;
    yourname.set(temp_str);
    myname.show();
    yourname.show();
    return 0;
}
```

D:\Edu-CPP\chap10\ex10-10.exe

```
Copy constructing.
zhang
zhang
wang
zhang
wang
Destructing.
Destructing.
```

键盘输入temp_str为wang

本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象

10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）



10.6 对象数组

10.7 对象指针和this指针

10.8 作业与实践



组合类的概念

◆ 一个类内嵌入其他类的对象作为数据成员，称为类的组合

■ 有类成员是另一个类对象

■ 可以在已有抽象的基础上实现更复杂的抽象

◆ 组合类的一般形式

```
class X
{
    ...
    类名1 成员名1;
    类名2 成员名2;
    ...
};
```

```
class Point
{
public:
    Point(int xx=0, int yy=0);
    Point(Point &p);
    void setXY(int xx, int yy) {x = xx; y = yy;};
    int getX() { return x; }
    int getY() { return y; }
private:
    int x, y;
};
```

```
class Line
{
public:
    Line(Point xp1, Point xp2);
    Line(Line &lin);
    double getLen() { return length; }
private:
    Point p1, p2;
    double length;
};
```

组合类对象成员的初始化



- ◆ 组合类**构造函数**不仅要负责本类成员数据初始化，还要负责对象成员初始化
- ◆ 组合类构造函数构造顺序
 - 先执行对象成员构造函数，再执行组合类构造函数
 - 如果有多个对象成员，按对象成员的定义顺序执行相应构造函数
- ◆ 如何调用对象成员带参的构造函数？
 - 利用**构造函数的初始化列表**，给带参构造函数传递参数

示例：设计二维平面的线段类，计算线段长度



```
class Point {
public:
    Point(int xx=0, int yy=0);
    Point(Point &p);
    void setXY(int xx, int yy) {x = xx; y = yy;};
    int getX() { return x; }
    int getY() { return y; }
private:
    int x, y;
};
```

```
Point::Point(int xx, int yy) {
    x = xx;    y = yy;
    cout << "Point Constructor(" << x << ", " << y << ")\n";
}

Point::Point(Point &p) {
    x = p.x;    y = p.y;
    cout << "Point Copy Constructor(" << x << ", " << y << ")\n";
}
```

```
class Line {
public:
    Line(Point xp1, Point xp2);
    Line(Line &lin);
    double getLen(){ return length; }
private:
    Point p1, p2;
    double length;
};
```

```
Line::Line(Point xp1, Point xp2) : p1(xp1), p2(xp2) {
    cout << "Calling constructor of Line\n";
    double x = p1.getX() - p2.getX();
    double y = p1.getY() - p2.getY();
    length = sqrt(x * x + y * y);
}

Line::Line (Line &lin) : p1(lin.p1), p2(lin.p2) {
    cout << "Calling copy constructor of Line\n";
    length = lin.length;
}
```



```
int main() {  
    Point myp1(1, 2), myp2(4, 5);  
    Line line(myp1, myp2);  
    Line line2(line);  
    cout << "The length of the line is: ";  
    cout << line.getLen() << "\n";  
    cout << "The length of the line2 is: ";  
    cout << line2.getLen() << "\n";  
    return 0; }
```

```
Line::Line(Point xp1, Point xp2) : p1(xp1), p2(xp2) {  
    cout << "Calling constructor of Line\n";  
    double x = p1.getX() - p2.getX();  
    double y = p1.getY() - p2.getY();  
    length = sqrt(x * x + y * y);  
}  
Line::Line (Line &lin) : p1(lin.p1), p2(lin.p2) {  
    cout << "Calling copy constructor of Line\n";  
    length = lin.length;  
}
```

D:\Edu-CPP\chap10\ex10-11.exe

```
Point Constructor(1, 2)  
Point Constructor(4, 5)  
Point Copy Constructor(4, 5)  
Point Copy Constructor(1, 2)  
Point Copy Constructor(1, 2)  
Point Copy Constructor(4, 5)  
Calling constructor of Line  
Point Copy Constructor(1, 2)  
Point Copy Constructor(4, 5)  
Calling copy constructor of Line  
The length of the line is: 4.24264  
The length of the line2 is: 4.24264
```

添加两个类的析构函数，分析程序中析构函数与构造函数的调用顺序

组合类构造函数的形式

X::X(形参表) : 成员1(参数表), 成员2(参数表), ...

成员（对象成员或非对象成员）初始化列表

{ 类的初始化 }

◆说明

- 当建立X类的对象时，先调用对象成员的构造函数，再执行X类的构造函数
 - 先客人（按照客人出场顺序），后自己
- 析构函数的调用顺序是先执行X类的析构函数，然后再调用对象成员的析构函数
 - 先自己，后客人
- 对象成员的构造函数的调用顺序，取决于这些对象成员的说明顺序，与其在初始化列表中的顺序无关，但二者顺序一致可提高效率

D:\Edu-CPP\chap10\ex10-11B.exe

```
Point Constructor(1, 2)
Point Constructor(4, 5)
Point Copy Constructor(4, 5)
Point Copy Constructor(1, 2)
Point Copy Constructor(1, 2)
Point Copy Constructor(4, 5)
Calling constructor of Line
Point Destructor(1, 2)
Point Destructor(4, 5)
Point Copy Constructor(1, 2)
Point Copy Constructor(4, 5)
Calling copy constructor of Line
The length of the line is: 4.24264
The length of the line2 is: 4.24264
Calling Destructor of Line
Point Destructor(4, 5)
Point Destructor(1, 2)
Calling Destructor of Line
Point Destructor(4, 5)
Point Destructor(1, 2)
Point Destructor(4, 5)
Point Destructor(1, 2)
```


成员初始化列表



- ◆ 若成员对象的类没有无参构造函数，则必须使用**成员初始化列表**初始化该成员对象，否则，自动调用并不存在的默认构造函数而编译失败

```
class Abc {
public:
    Abc(int x, int y, int z) { a = x, b = y, c = z; }
private:
    int a, b, c;
};

class MyClass {
public:
    MyClass() : abc(1, 2, 3) { }
private:
    Abc abc;
};
```

```
class Abc {
public:
    Abc(int x, int y, int z) { a = x, b = y, c = z; }
private:
    int a, b, c;
};

class MyClass {
public:
    //MyClass() : abc(1, 2, 3) { }
    MyClass() { }
private:
    Abc abc;
};
```

Compile Log ☒ Debug ☐ Find Results ☒ Close

Message

In constructor 'MyClass::MyClass()':

[Error] no matching function for call to 'Abc::Abc()'



◆ **const**成员或引用类型的成员，必须使用成员初始化列表（不能用赋值实现初始化）

```
class Screen {  
private:  
    const int size; // 只能在类中使用，不同对象中其值可以不同  
    long p;  
    long &price;  
public:  
    Screen(int s, long m) : size(s), price(p)  
    {  
        p = m; //利用赋值方式赋成员p初值  
    }  
    .....  
}
```

若不是类的成员，则须在定义时初始化；类成员不允许在定义时初始化



◆ 利用初始化列表初始化数据成员与对数据成员赋值的区别

- 对于int、float等基本数据类型、复合类型（指针、引用），二者在性能和结果上都是一样的

```
Point::Point(int xx, int yy) {  
    x = xx;    y = yy;  
}
```

```
Point::Point(int xx, int yy) : x(xx), y(yy)  
{  
}
```

- 对于类类型，结果相同，性能上有很大的差别：若使用初始化列表，成员对象在**进入构造函数的函数体之前**，即在初始化列表处调用拷贝构造函数**构造并初始化对象**，否则，调用无参构造函数构造对象，进入函数体之后再对已构造的类对象进行赋值才能完成，性能相对较差

```
Line::Line(Point xp1, Point xp2) : p1(xp1), p2(xp2) {  
    cout << "Calling constructor of Line\n";  
    double x = p1.getX() - p2.getX();  
    double y = p1.getY() - p2.getY();  
    length = sqrt(x * x + y * y);  
}
```

```
Line::Line(Point xp1, Point xp2){  
    p1 = xp1; // 相当于 p1.x = p2.x, p1.y = p2.y;  
    p2 = xp2;  
    double x = p1.getX() - p2.getX();  
    .....  
}
```



前向引用声明

◆ 一个类在定义之前被引用，需用前向引用声明

```
class B;           // 前向引用声明，只引入一个标识符，无细节
class A
{
public:
    B obj_b;        // error，无完整声明，不能声明该类对象
    B *p;           // ok
    void f1(B b);    // ok
    void f2(B b){....} // error，不能在内联函数中使用该类的对象
};

class B
{
public:
    void g(A a);
};
```

只能使用被声明的符号，而不能涉及类的任何细节；在所有类定义完之后，再实现所有的函数

本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象

10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）

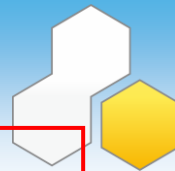
10.6 对象数组



10.7 对象指针和this指针

10.8 作业与实践

对象数组



◆ 对象数组的定义

类名 数组名[元素个数]; `Point a[2];`

◆ 访问方法：通过下标访问

数组名[下标].成员名 `a[1].getX();`

◆ 对象数组元素的构造

■ 每个元素对象被创建时，都会调用类构造函数初始化该对象

➢ 每个元素对象的初值要求相同时，可声明具有默认形参值的构造函数

➢ 每个元素对象的初值要求不同时，需声明带形参的构造函数

➢ 若没有为数组元素指定显式初始值，数组元素则使用默认值初始化（无参构造函数）

■ 示例：利用带参构造函数初始化元素 `Point a[2] = { Point(1, 2), Point(3, 4) };`

◆ 对象数组元素的析构

■ 当数组中每一个对象元素被删除时，系统都要调用一次析构函数

```
class Point {  
public:  
    Point(int xx=0, int yy=0) { x = xx; y = yy; }  
    Point(Point &p) { x = p.x; y = p.y; }  
    void setXY(int xx, int yy) {x = xx; y = yy;};  
    int getX() { return x; }  
    int getY() { return y; }  
private:  
    int x, y;  
};
```

匿名对象

对象数组示例



```
class Point {
public:
    Point() { x = y = 0; cout << "Default Constructor called.\n"; }
    Point(int x, int y) : x(x), y(y) { cout << "Constructor called.\n"; }
    ~Point() { cout << "Destructor called(" << x << ", " << y << ").\n"; }
    void move(int x, int y){
        this->x = x; (*this).y = y;
        cout << "Moving the point to (" << x << ", " << y << ")\n";
    }
    int getX() const { return x; }
    int getY() const { return y; }
private:
    int x, y;
};

int main() {
    Point a[2];
    Point b[2] = { Point(1, 2), Point(3, 4) };
    for(int i = 0; i < 2; i++) a[i].move(i + 10, i + 20);
    return 0;
}
```

D:\Edu-CPP\chap10\ex10-13.exe

```
Default Constructor called.
Default Constructor called.
Constructor called.
Destructor called(1, 2).
Constructor called.
Destructor called(3, 4).
Moving the point to (10, 20)
Moving the point to (11, 21)
Destructor called(3, 4).
Destructor called(1, 2).
Destructor called(11, 21).
Destructor called(10, 20).
```

本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象

10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）

10.6 对象数组

10.7 对象指针和this指针



10.8 作业与实践

对象指针



◆对象指针的定义形式

类名 *对象指针名;

■例如:

```
Point a(5, 10);
```

```
Point *ptr;
```

```
ptr = &a;
```

◆利用指针访问对象成员

对象指针名->成员名

■例如:

ptr->getX();等价于(*ptr).getX();

```
class Point
```

```
{
```

```
public:
```

```
    Point(int xx=0, int yy=0) { x = xx; y = yy; }
```

```
    Point(Point &p) { x = p.x; y = p.y; }
```

```
    void setXY(int xx, int yy) { x = xx; y = yy; }
```

```
    int getX() { return x; }
```

```
    int getY() { return y; }
```

```
private:
```

```
    int x, y;
```

```
};
```

```
int main()
```

```
{
```

```
    Point myp1(4, 5);
```

```
    Point *p = &myp1;
```

```
    cout << p->getX() << ", " << (*p).getY() << '\n';
```

```
    cout << myp1.getX() << ", " << myp1.getY() << '\n';
```

```
    return 0;
```

```
}
```

C:\C++Examp\examp10\ex10-13.exe

4, 5
4, 5

用new动态创建对象



- ◆ 可以用new运算符动态创建对象。用new运算符建立对象时，同样也要自动调用构造函数，以便完成对象数据成员的初始化

例如：

```
MyClass *pClass = new MyClass(9);  
MyClass *pClass2 = new MyClass;
```

调用相应的构造函数（此处为有参/无参构造函数）动态创建并初始化对象

- ◆ 对于用new运算符动态创建的对象，在创建对象时调用相应的构造函数，只有用delete释放对象时，才调用析构函数

```
int main( ) {  
    // gFunc( );  
    MyClass *pClass = new MyClass(9);  
    MyClass *pClass2 = new MyClass;  
    delete pClass;  
    //delete pClass2;  
    cout << "退出main( )" << endl;  
    return 0;  
}
```

C:\C++Examp\examp10\ex10-07.exe
析构函数中value=9
退出main()

教材第374页第14行：【当程序结束时，对象会被销毁。也可以使用关键字delete显式销毁对象。】应改为必须用操作符delete显式销毁对象，否则内存泄漏、析构函数未执行。

对象的this指针



◆ 成员函数为类的所有对象共享，如何知道哪个对象在调用成员函数？

◆ 对象的this指针

■ C++为每个对象提供一个指针this，记录该对象的地址

- 创建对象时，编译器将this指针初始化为指向该对象，即this是指向对象自身的一个指针
- this指针是一个**指针常量**，不能在程序中修改或赋值
- this指针是一个局部变量，其作用域仅限于一个对象内部，在运行时有效

■ 调用类的**非静态**成员函数时，this指针作为一个**隐式参数**传递给该成员函数

- 不同的对象调用同一个成员函数时，编译器依据成员函数的this指针所指向，确定哪一个对象在调用该成员函数

```
void Clock::setTime(int h, int m, int s){  
    hour = h;  
    minute = m;  
    second = s;  
}
```

被编译器
改写为



```
void Clock::setTime(Clock *const this, int h, int m, int s) {  
    this->hour = h;  
    this->minute = m;  
    this->second = s;  
}
```

当某个对象调用成员函数时，this指针便指向该对象

Clock c1, c2;
c1.setTime(1, 2, 3);//编译器将其编译成:
Clock::setTime(**&c1**, 1, 2, 3);



◆ 利用this指针访问被屏蔽的数据成员

```
class Point {  
public:  
    Point() { x = y = 0; cout << "Default Constructor called.\n"; }  
    Point(int x, int y) : x(x), y(y) { cout << "Constructor called.\n"; }  
    ~Point() { cout << "Destructor called(" << x << ", " << y << ").\n"; }  
    void move(int x, int y){  
        this->x = x;  (*this).y = y;  
        cout << "Moving the point to (" << x << ", " << y << ")\n";  
    }  
    int getX() const { return x; }  
    int getY() const { return y; }  
private:  
    int x, y;  
};
```

常函数，声明不能利用本函数修改数据成员



示例：模拟数字时钟

◆思路

■对所有的时钟对象进行抽象：哪些数据？哪些功能（函数）？

```
class Clock
{
    public:
        Clock(int h=0, int m=0, int s=0);
        void setTime(int h, int m, int s);
        void showTime();
    private:
        int hour, minute, second;
};
```

时分秒的改变需要循环计数器



◆ 循环计数器类

■ 数据抽象

```
int min;    // 最小值  
int max;    // 最大值  
int current; // 当前值
```

■ 功能抽象

```
void setMode(int min, int max); // 设置循环计数器的上、下限  
void setValue(int value);      // 设置循环计数器的当前值  
int getValue();                // 查询循环计数器的当前值  
void increment();              // 循环计数器加1  
void decrement();              // 循环计数器减1
```

```
class CircularNumber
{
public:
    CircularNumber(int min, int max, int value);
    void setMode(int min, int max);           // 设置循环计数器的上、下限
    void setValue(int value){ current = value; } // 设置循环计数器的当前值
    int getValue(){ return current; }         // 查询循环计数器的当前值
    void increment();                         // 循环计数器加1
    void decrement();                        // 循环计数器减1
private:
    int min;    // 最小值
    int max;    // 最大值
    int current; // 当前值
};
```

```
CircularNumber::CircularNumber(int min, int max, int value)
{
    this->min = (min <= max) ? min : max;
    this->max = (min <= max) ? max : min;
    if (value < this->min)
        current = this->min;
    else
    {
        if (value > this->max)
            current = this->max;
        else
            current = value;
    }
}

void CircularNumber::setMode(int min, int max)
{
    this->min = min;
    this->max = max;
}
```

```
void CircularNumber::increment()
{
    int mode = max - min + 1;
    current = ((current - min) + 1) % mode + min;
}

void CircularNumber::decrement()
{
    int mode = max - min + 1;
    current = ((current - min) - 1 + mode) % mode + min;
}
```




◆ 时钟类

```
class Clock
{
public:
    Clock(int h=23, int m=59, int s=59);
    void setTime( int h, int m, int s);
    void showTime();
    void update();
private:
    CircularNumber hour;
    CircularNumber minute;
    CircularNumber second;
};
```

```
Clock::Clock(int h, int m, int s) :
    hour(0, 23, h), minute(0, 59, m), second(0, 59, s){
}
void Clock::showTime() {
    cout << "\r" << hour.getValue() << ":" << minute.getValue()
        << ":" << second.getValue();
}
void Clock::setTime(int h, int m, int s) {
    hour.setValue(h);
    minute.setValue(m);
    second.setValue(s);
}
void Clock::update() {
    second.increment();
    if(second.getValue() == 0) {
        minute.increment();
        if(minute.getValue() == 0) hour.increment();
    }
}
```

```
int main()
{
    Clock rolex(4, 15, 55);
    cout<<"Rolex: \n";
    for(int i = 0; i < 100; i++)
    {
        rolex.update();
        rolex.showTime();
    }
    return 0;
}
```

 C:\C++Examp\examp

```
Rolex:
4:15:56
4:15:57
4:15:58
4:15:59
4:16:0
4:16:1
```

改进：取当前时间

```
int main()
{
    time_t tt = time(NULL);
    tm* t= localtime(&tt);
    Clock rolex(t->tm_hour,
                t->tm_min,
                t->tm_sec);
    cout << "Rolex: \n";
    for(int i = 0; i < 100; i++)
    {
        rolex.update();
        rolex.showTime();
    }
    return 0;
}
```

改进：空循环实现延迟

```
int main()
{
    time_t tt = time(NULL);
    tm* t= localtime(&tt);
    Clock rolex(t->tm_hour, t->tm_min, t->tm_sec);
    cout << "Rolex: \n";
    for(int i = 0; i < 100; i++)
    {
        clock_t start = clock();
        while(clock() - start < CLOCKS_PER_SEC);
        rolex.update();
        rolex.showTime();
    }
    return 0;
}
```

 C:\C++Examp\examp10\ex10-14.exe

```
Rolex:
23:15:09
```

纯粹是为了展示面向对象技术，可以sleep(1000)读系统时间并显示

本章主要内容



10.1 面向过程程序设计与面向对象程序设计

10.2 类和对象

10.3 构造函数和析构函数

10.4 深拷贝和浅拷贝

10.5 类的组合（对象成员）

10.6 对象数组

10.7 对象指针和this指针

10.8 作业与实践



第10章作业



(1)设计一个名为MyPoint的类，表示直角坐标系中一个点，类包含：

- 两个表示坐标的数据成员x和y；
- 一个无参构造函数，创建一个点(0, 0)；
- x和y的get函数；
- 一个名为distance的函数，返回当前点与另一个给定的MyPoint类型点之间距离

编写测试程序创建两个点(0, 0)和(10, 30.5)，并输出此两点之间的距离

(2)定义秒表类StopWatch，类包含：

- 私有数据成员start_time和end_time，及相应的get函数；
- 无参构造函数，利用当前时间初始化start_time；
- 成员函数start()重置start_time为当前时间；
- 成员函数stop()设置end_time为当前时间；
- 成员函数getElapsedTime()返回该秒表流逝的时间，以毫秒为单位

编写测试程序，测量选择排序算法排序100000个数消耗的时间



(3)定义表示偶数的类**EvenNumber**，类包含：

- int型数据成员value，表示对象存储的整型数；
- 一个无参构造函数，为数值0创建EvenNumber对象；
- 一个构造函数，为特定数值创建EvenNumber对象；
- 成员函数getValue()返回对象存储的int型值；
- 成员函数getNext()返回本对象表示的数值的下一个偶数所对应的EvenNumber对象；
- 成员函数getPrevious()返回本对象表示的数值的前一个偶数所对应的EvenNumber对象

编写测试程序为数值16创建EvenNumber对象，调用getNext()和getPrevious()获取并输出相应偶数值

(4)设计一个时钟类**Clock**

- 数据成员：时(int hour)，分(int minute)，秒(int second)；
- 成员函数：构造函数Clock(int hour=0, int minute = 0, int second =0)，设置函数void setTime(int h, int m, int s)，显示时间函数void showtime()，计算本对象时刻与一给定时刻的间隔秒数函数int interval(int h, int m, int s)（形参也可以是Clock）

编写主函数测试Clock类



◆(5) 定义一个整数类，其功能包括：

- 无参构造函数，可以不赋值，或者赋固定值，并输出“无参构造函数被调用”；
- 有参构造函数，实现对整数的初始化，并输出“有参构造函数被调用”；
- 拷贝构造函数，实现用整数类对象为其初始化，并输出“拷贝构造函数被调用”；
- 析构函数，在析构函数中输出“析构函数被调用”
- 分别定义成员函数，实现如下功能：给整型数赋值，读取整型数，质数判断、回文数判断、计算位数

编写主函数测试

◆(6)定义复数类Complex，使得下面代码能够工作

```
Complex c1(3, 5);    //用复数3+5i初始化c1
Complex c2 = 4.5;    //用实数4.5初始化
c1.add(c2);          //将c1和c2相加保存在c1
c1.show();            //将c1输出
```



(7)定义一个Circle类

- 数据成员：半径radius，圆心位置position（position是Point类的类对象），静态变量记录圆的个数；
- 函数成员：有参构造函数，实现对Point有参构造函数传递参数；计算面积函数；设置半径和位置函数；显示半径和位置函数；静态函数显示圆的个数

Point类：有参构造函数，设置位置函数，显示位置函数

注意：为了安全，哪些函数可以设置为常函数

(8)设计一个学生类Student

- 数据成员：学号(char id[11])，姓名(char * name，英文字符串)，数学(float math)、物理(float physics)、英语(float english)成绩；
- 成员函数有：构造函数、析构函数、设置学生信息函数、计算总成绩函数、输出学生全部信息函数和获取学生学号函数

编写主函数：创建有5个学生的对象数组并进行学生信息的设置；计算每个学生的总成绩；求出5人中的最高总分；输入一个学号，输出该同学的全部信息